

Notas Completas: Árboles, Computación Evolutiva y Optimización

Índice

1. Estructura de Datos: Árboles

1.1. Definición y Conceptos Básicos

Los árboles son estructuras de datos no lineales fundamentales en ciencias de la computación. Sirven para representar información jerárquica, direcciones o etiquetas de una manera organizada.

Definición formal recursiva: Un árbol es un conjunto finito de uno o más nodos donde existe un nodo especial llamado **raíz**. Los nodos restantes se dividen en $m \geq 0$ conjuntos disjuntos $T_1, \dots, T_m$, donde cada uno de estos conjuntos es a su vez un árbol (llamados subárboles de la raíz). Cualquier nodo es la raíz de un subárbol que consiste en él y los nodos por debajo de él.

Ejemplos en la vida cotidiana:

- La organización de una empresa (organigrama).
- El árbol genealógico de una persona.
- La organización de torneos deportivos (tabla de partidos).

Aplicaciones computacionales principales:

- Realización de ordenaciones y búsquedas de datos.
- Asignación de bloques de memoria de tamaño variable.
- Organización de tablas de símbolos en compiladores.
- Representación de tablas de decisión.
- Solución de juegos y prueba de teoremas.

1.2. Terminología de Árboles

Para entender y manipular árboles, es vital conocer su anatomía básica:

- **Nodo o Vértice:** Elemento del árbol que puede tener un nombre e información asociada.
- **Raíz:** El primer nodo de un árbol. Su profundidad siempre es 0.
- **Arcos o Ramas:** Las flechas o líneas que conectan un nodo con otro.
- **Hojas (Nodos Terminales):** Nodos que no tienen hijos (no conducen a ningún otro nodo).
- **Nodos Internos (No Terminales):** Cualquier nodo que no sea una hoja.
- **Relación Padre-Hijo:** Si un nodo n_1 tiene una rama hacia n_2 , se dice que n_2 es hijo de n_1 .
- **Camino:** Una lista de ramas contiguas que van de un nodo a otro.
- **Nivel de un Nodo:** La longitud del camino que lo conecta al nodo raíz.
- **Altura de un Nodo:** El máximo largo de camino desde ese nodo hasta alguna hoja.
- **Bosque:** Un conjunto de árboles.

1.3. Árboles Binarios (AB)

Un árbol binario es aquel en el que ningún nodo puede tener más de dos subárboles (puede tener cero, uno o dos hijos: izquierdo y derecho).

- **Árbol Equilibrado:** Se determina mediante su Factor de Equilibrio (FE), que es la diferencia entre la altura del subárbol derecho y el izquierdo:

$$FE = h_{SD} - h_{SI}$$

- **Árbol Binario Completo:** Un árbol de profundidad N donde cada nivel (del 0 al $N - 1$) tiene su conjunto de nodos completamente lleno.

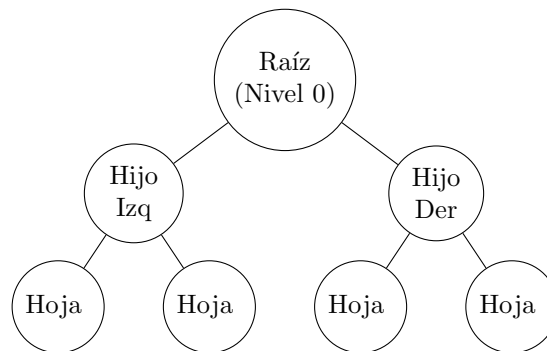
- **Profundidad:** La fórmula para obtener la profundidad de un árbol completo a partir de su número de nodos (n) es:

$$h = \lfloor \log_2 n \rfloor + 1$$

Representación ligada en C:

```
typedef struct _nodo {
    int info;
    struct _nodo *izq;
    struct _nodo *der;
} tiponodo;
typedef tiponodo *Arbol;
```

Diagrama de un Árbol Binario:



1.4. Recorridos en Árboles Binarios

- **Preorden:** Visitar la raíz → Recorrer subárbol izquierdo → Recorrer subárbol derecho.
- **Inorden:** Recorrer subárbol izquierdo → Visitar la raíz → Recorrer subárbol derecho.
- **Postorden:** Recorrer subárbol izquierdo → Recorrer subárbol derecho → Visitar la raíz.

1.5. Árboles Binarios de Búsqueda (BST)

Un conjunto de nodos T es un árbol binario de búsqueda si es vacío, o si para cada nodo n :

1. La llave de búsqueda de n es mayor que todas las llaves de su subárbol izquierdo (T_I).
2. La llave de búsqueda de n es menor que todas las llaves de su subárbol derecho (T_D).
3. Ambos T_I y T_D son también árboles binarios de búsqueda.

1.6. Operaciones Básicas en un BST

Para que un Árbol Binario de Búsqueda (BST) sea útil, debe soportar tres operaciones fundamentales. La eficiencia de estas operaciones es de $O(\log n)$ en promedio, pero puede degradarse a $O(n)$ si el árbol está desbalanceado (semejante a una lista ligada).

- **Búsqueda:** Se compara el valor buscado con la raíz. Si es igual, la búsqueda termina. Si es menor, se busca recursivamente en el subárbol izquierdo; si es mayor, en el subárbol derecho.
- **Inserción:** Se realiza una búsqueda hasta llegar a una hoja (un puntero nulo). El nuevo nodo se inserta en esa posición, respetando la regla de orden del BST.
- **Eliminación (El caso más complejo):** Tiene tres escenarios posibles:
 1. **El nodo a eliminar es una hoja:** Simplemente se borra y se actualiza el puntero de su padre a nulo.

2. **El nodo tiene un solo hijo:** Se elimina el nodo y se reemplaza por su único hijo (el abuelo .^adopta.^al nieto).
3. **El nodo tiene dos hijos:** Se busca su *sucesor inorden* (el nodo más pequeño del subárbol derecho) o su *predecesor inorden* (el nodo más grande del subárbol izquierdo). Se copia el valor de este sucesor al nodo que queremos eliminar, y luego se elimina el sucesor original.

1.7. Árboles Auto-balanceables: Árboles AVL

Para evitar el peor caso de rendimiento $O(n)$ en un BST, se desarrollaron los árboles auto-balanceables. El primer modelo propuesto fue el **Árbol AVL** (por sus inventores Adelson-Velsky y Landis).

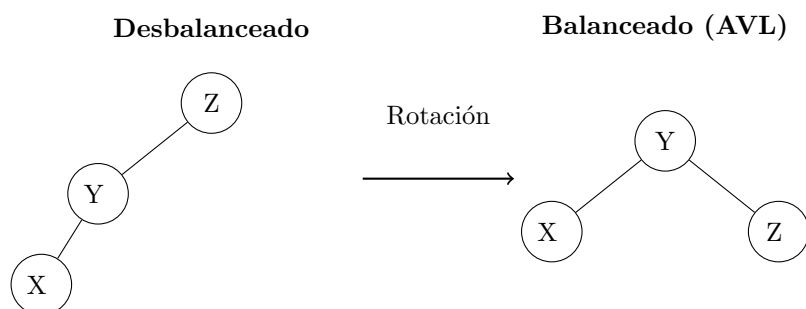
Un árbol es AVL si, para cada nodo, la diferencia de alturas entre su subárbol derecho y su subárbol izquierdo (el Factor de Equilibrio, FE) es exactamente -1 , 0 o 1 .

$$FE = h_{SD} - h_{SI} \in \{-1, 0, 1\}$$

Rotaciones: Cuando se inserta o elimina un nodo y un subárbol rompe la regla del FE (por ejemplo, el FE se vuelve 2 o -2), el árbol debe reestructurarse mediante *rotaciones* para restaurar el balance:

- **Rotación Simple a la Derecha (LL):** Se aplica cuando el desequilibrio ocurre por insertar un nodo en el subárbol izquierdo del hijo izquierdo.
- **Rotación Simple a la Izquierda (RR):** Se aplica cuando el desequilibrio ocurre en el subárbol derecho del hijo derecho.
- **Rotaciones Dobles (LR y RL):** Combinaciones de las anteriores cuando el desequilibrio ocurre en posiciones internas (en zigzag).

Diagrama de Rotación Simple a la Derecha:



1.8. Montículos (Heaps)

Un montículo es una estructura de datos basada en árboles binarios que satisface dos propiedades fundamentales:

1. **Propiedad de forma:** Es un árbol binario completo en todos sus niveles, excepto posiblemente el último, el cual se llena de izquierda a derecha.
2. **Propiedad de orden:** Dependiendo del tipo de montículo, la relación entre padres e hijos está estrictamente definida.

Tipos de Heaps:

- **Min-Heap:** El valor de cada nodo padre es menor o igual al valor de sus hijos. La raíz siempre contiene el elemento más pequeño.
- **Max-Heap:** El valor de cada nodo padre es mayor o igual al valor de sus hijos. La raíz siempre contiene el elemento más grande.

Los montículos son la base para algoritmos de ordenamiento muy eficientes como el **Heapsort** ($O(n \log n)$) y son la estructura de datos ideal para implementar **Colas de Prioridad**, utilizadas frecuentemente en algoritmos de grafos (como Dijkstra) y planificación de procesos del sistema operativo.

2. Computación Evolutiva

2.1. Teoría de Esquemas y Paralelismo Implícito

Los algoritmos genéticos (AG) funcionan descubriendo y enfatizando “bloques constructores” de soluciones de forma altamente paralela.

Cualquier cadena de bits de longitud L pertenece a 2^L esquemas diferentes. Un esquema es un patrón de similitud que describe a un conjunto de individuos usando el alfabeto $\{0, 1, *\}$.

El **paralelismo implícito** ocurre porque, mientras el algoritmo evalúa las aptitudes de N cadenas en la población en una generación, está estimando simultáneamente las aptitudes de un número muchísimo mayor de esquemas.

- **Orden de esquema** $O(H)$: Número fijo de alelos (ceros o unos) presentes en el patrón.
- **Longitud** $d(H)$: Distancia física entre el primer y el último elemento fijo del esquema.

La población esperada de un esquema H en la siguiente generación $(t + 1)$ se calcula como:

$$m(H, t + 1) = m(H, t) \times \frac{f(H)}{\hat{f}}$$

Donde $\hat{f}$ es el promedio de aptitud de la población.

Efecto de la Cruza y Mutación en los Esquemas:

- **Probabilidad de destrucción por cruza:**

$$P_{\text{destrucción}}(H) = \frac{d(H)}{l - 1}$$

- **Probabilidad de supervivencia a la mutación:**

$$(1 - P_m)^{O(H)}$$

2.2. Técnicas de Codificación

John Holland determinó que la **codificación binaria** (usando el alfabeto $\{0, 1\}$) era la más conveniente teóricamente.

Decodificación a variables reales: Para convertir una subcadena binaria a un valor numérico real en un rango, se utiliza:

$$x_i = u_i + \frac{v_i - u_i}{2^{l_i} - 1} \left(\sum_{j=0}^{l_i-1} a_j 2^j \right)$$

- **Código Gray:** La codificación binaria normal a menudo no mapea bien el espacio de búsqueda debido al “risco de Hamming”. El código Gray soluciona esto aplicando operaciones XOR a los bits consecutivos.
- **Codificación Desordenada (Linkage problem):** Para evitar que la cruza destruya bloques constructores que están muy separados en la cadena, se almacena de forma conjunta la posición del gen y su valor.

2.3. Programación Genética (Codificación de Árbol)

Este enfoque, establecido por John Koza, evoluciona programas de computadora representados como árboles en lugar de cadenas de bits.

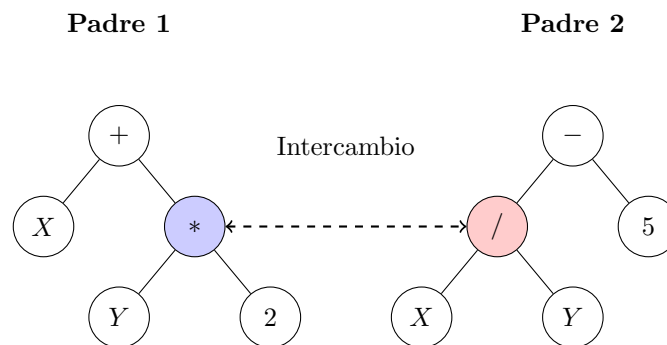
Componentes del árbol:

1. **Nodos internos (Funciones):** Operadores aritméticos (+, −, *, /), funciones matemáticas (sin, exp, ln), operadores booleanos (AND, OR, NOT), condicionales (If-Then-Else), ciclos y funciones recursivas.
2. **Terminales (Hojas):** Variables del problema o constantes numéricas.

Operaciones Genéticas en Árboles:

- **Cruza:** Se seleccionan puntos de cruce al azar en dos árboles padres y se intercambian los subárboles completos a partir de esos puntos.
- **Mutación:** Se selecciona un punto al azar en el árbol y se reemplaza todo el subárbol debajo de él con un nuevo subárbol generado aleatoriamente.

Diagrama de Cruza en Programación Genética:



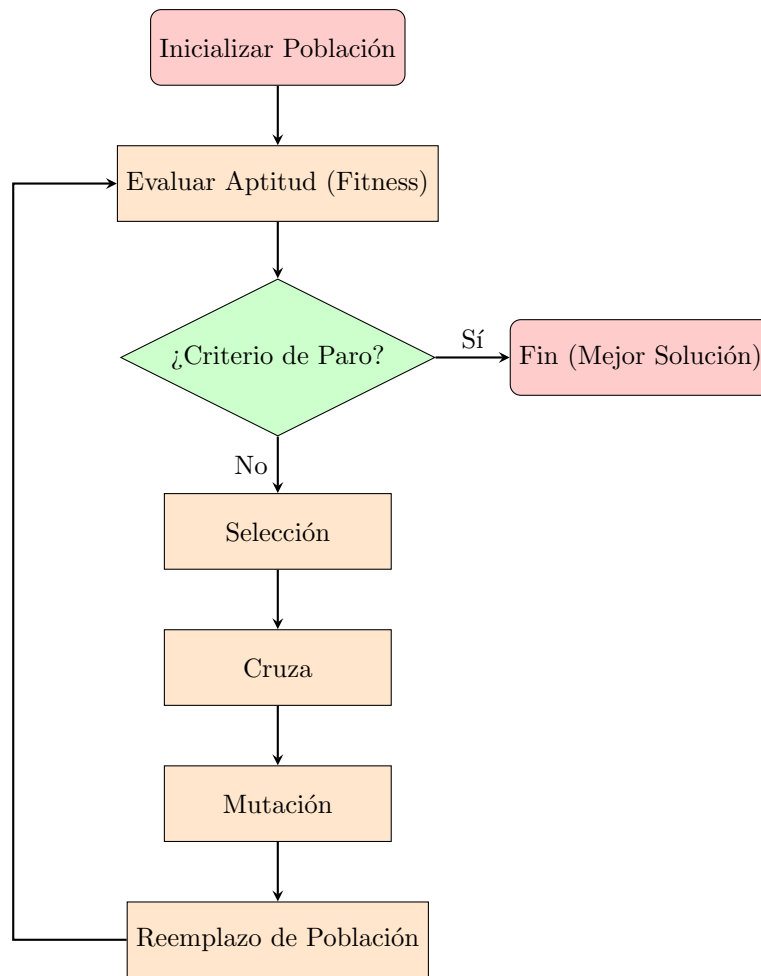
2.4. El Algoritmo Genético Clásico

Función Objetivo vs. Función de Aptitud (Fitness): Es vital distinguir entre ambas. La **función objetivo** es la ecuación matemática pura que describe el problema (puede ser minimizar costos o maximizar ganancias, y puede tener valores negativos). La **función de aptitud** (f) transforma esa función objetivo en una medida de “supervivencia” o desempeño, la cual típicamente debe ser estrictamente positiva para ser evaluada correctamente por el algoritmo.

Pseudocódigo del Algoritmo Genético: El proceso iterativo básico de un algoritmo genético sigue esta estructura:

1. INICIALIZAR población $P(t=0)$ de tamaño N aleatoriamente.
2. EVALUAR la aptitud $f(x)$ de cada individuo en $P(t)$.
3. MIENTRAS (no se cumpla criterio de paro) HACER:
 - a. SELECCIONAR padres de $P(t)$ basados en su aptitud.
 - b. CRUZAR padres para generar descendencia.
 - c. MUTAR la descendencia con una probabilidad P_m .
 - d. EVALUAR la aptitud de los nuevos individuos.
 - e. REEMPLAZAR la población $P(t)$ para formar $P(t+1)$.
 - f. $t = t + 1$
4. FIN MIENTRAS
5. RETORNAR el mejor individuo encontrado.

Diagrama de Flujo del Algoritmo Genético:



2.5. Métodos de Selección y Criterios de Paro

La fase de selección determina cuáles individuos sobrevivirán y se reproducirán. Se debe mantener una presión selectiva adecuada: mucha presión lleva a convergencia prematura (estancamiento en un mínimo local), y muy poca presión hace que el algoritmo se comporte como una búsqueda aleatoria.

Principales Métodos de Selección:

- **Ruleta (Proporcional a la aptitud):** A cada individuo se le asigna un segmento de una ruleta cuyo tamaño es proporcional a su valor de aptitud. Los mejores individuos tienen más probabilidades de ser seleccionados. La probabilidad de seleccionar al individuo i es:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}$$

- **Torneo:** Se seleccionan k individuos al azar de la población y compiten entre sí. El que tenga la mejor aptitud gana y es seleccionado. Es muy popular porque es eficiente y no requiere escalar la función de aptitud.
- **Jerarquía (Ranking):** Se ordena a la población de peor a mejor según su aptitud. La probabilidad de selección se asigna según la posición (rango) en la lista, no por el valor exacto de la aptitud. Esto evita que un individuo "súper apto" domine la población rápidamente en las primeras generaciones.

Criterios de Paro Comunes: El algoritmo debe detenerse en algún momento. Las condiciones típicas son:

1. **Número máximo de generaciones:** Se alcanza un límite de iteraciones (T_{max}).
2. **Convergencia de la población:** Cuando la mayoría de los individuos en la población son idénticos o muy similares, lo que significa que el algoritmo ya no está explorando soluciones nuevas.

3. **Umbral de aptitud (Fitness threshold):** Cuando se encuentra una solución cuya aptitud es igual o mejor que un valor meta predefinido.

3. Técnicas Clásicas de Optimización

3.1. Introducción

La optimización consiste simplemente en encontrar la mejor solución posible entre las alternativas disponibles dadas unas circunstancias específicas. Todo problema de optimización consta de tres componentes principales:

1. Una función objetivo a maximizar o minimizar.
2. Variables independientes.
3. Restricciones del problema.

Las técnicas clásicas asumen que la función objetivo es continua, tiene derivadas continuas y es dos veces diferenciable.

3.2. Condiciones Necesarias y Suficientes

Para garantizar que un punto encontrado es un óptimo, se basan en teoremas de cálculo:

Funciones de una variable $f(x)$:

- **Condición Necesaria:** Si la función tiene un mínimo local en $x = x^*$, entonces la primera derivada es cero: $f'(x) = 0$.
- **Condición Suficiente:** Si $f'(x^*) = \dots = f^{(n-1)}(x^*) = 0$ pero la n -ésima derivada $f^{(n)}(x^*) \neq 0$:
 - Si $f^{(n)}(x^*) > 0$ y n es par, es un **mínimo**.
 - Si $f^{(n)}(x^*) < 0$ y n es par, es un **máximo**.

Funciones multivariantes $f(X)$:

- **Condición Necesaria:** El gradiente en el punto óptimo debe ser cero: $\nabla f(X^*) = 0$.
- **Condición Suficiente:** Evaluando la Matriz Hessiana en X^* :
 - Si es definida positiva, el punto es un mínimo relativo.
 - Si es definida negativa, el punto es un máximo relativo.
 - Si no es ninguna de las dos, se trata de un punto de silla.

3.3. Técnicas Numéricas Basadas en Gradiente

- **Descenso Más Pronunciado (Steepest Descent):** Introducida por Cauchy. Comienza en un punto inicial y elige como dirección de búsqueda el gradiente negativo:

$$S_i = -\nabla f(X_i)$$

Luego, busca el tamaño de paso óptimo λ_i^* que minimice la función en esa dirección: $f(X_i + \lambda_i^* S_i)$.

- **Método de Newton:** Utiliza tanto la primera como la segunda derivada para determinar la dirección de búsqueda:

$$S_i = -H_i^{-1} \nabla f_i$$

Converge muy rápido para funciones cuadráticas, pero requiere calcular la inversa de la Matriz Hessiana (H_i^{-1}).

3.4. Técnicas de Eliminación

Se utilizan generalmente para funciones unidimensionales que son discontinuas o no diferenciables (funciones unimodales).

- **Método de Eliminación de Fibonacci:** Utiliza la secuencia de Fibonacci para seleccionar puntos de prueba estratégicos e ir reduciendo el intervalo de incertidumbre:

$$L_2^* = \frac{F_{n-2}}{F_n} L_0$$

- **Método de la Sección Dorada:** Inspirado en la proporción áurea ($\gamma \approx 1,618$). Es similar al de Fibonacci, pero utiliza un factor de reducción constante:

$$L_2^* = \frac{L_0}{\gamma^2} = 0,382 L_0$$

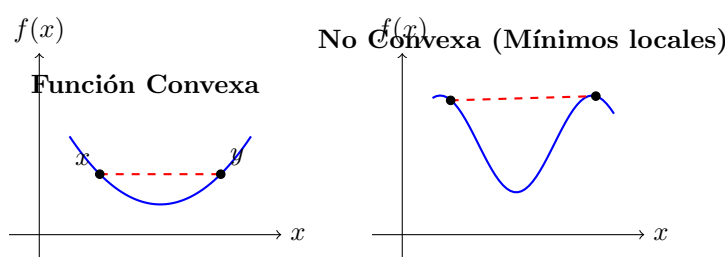
3.5. Convexidad

Un concepto vital en la teoría de optimización es la convexidad. La principal ventaja de una función convexa es que **cualquier mínimo local es también un mínimo global**, lo que garantiza que las técnicas basadas en gradiente encuentren la solución óptima absoluta sin quedarse atrapadas.

Definición matemática: Una función $f(x)$ es convexa si, para cualesquiera dos puntos x, y en su dominio y cualquier factor $t \in [0, 1]$, el segmento de recta que une esos puntos siempre se mantiene por encima o sobre la curva de la función:

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$$

Diagrama de Convexidad:



3.6. Optimización con Restricciones

Los problemas del mundo real raramente son irrestrictos; generalmente están limitados por recursos, leyes de la física o presupuestos. Las técnicas clásicas requieren adaptaciones mayores para resolverlos.

1. Multiplicadores de Lagrange (Restricciones de Igualdad): Se utilizan cuando el problema tiene la forma: Minimizar $f(x)$ sujeto a $h_i(x) = 0$.

Se construye una nueva función combinada llamada **Función Lagrangiana**, agregando variables artificiales λ_i (los multiplicadores):

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i h_i(x)$$

El óptimo se encuentra resolviendo $\nabla \mathcal{L}(x, \lambda) = 0$, lo que implica que en el punto óptimo, el gradiente de la función objetivo es paralelo al gradiente de la restricción.

2. Condiciones KKT (Restricciones de Desigualdad): Nombradas por Karush, Kuhn y Tucker, son una generalización de los multiplicadores de Lagrange para problemas del tipo: Minimizar $f(x)$ sujeto a $g_j(x) \leq 0$ y $h_i(x) = 0$.

Para que un punto x^* sea un óptimo garantizado (asumiendo que las funciones son diferenciables), debe satisfacer estrictamente las siguientes condiciones KKT:

- **Estacionariedad:** El gradiente del Lagrangiano generalizado debe ser cero:
$$\nabla f(x^*) + \sum \mu_j \nabla g_j(x^*) + \sum \lambda_i \nabla h_i(x^*) = 0$$

- **Factibilidad primal:** El punto no debe violar ninguna restricción original del problema: $g_j(x^*) \leq 0$ y $h_i(x^*) = 0$.
- **Factibilidad dual:** Los multiplicadores asociados a las desigualdades deben ser positivos: $\mu_j \geq 0$.
- **Holgura complementaria:** $\mu_j g_j(x^*) = 0$. Esto significa que si una restricción de desigualdad no está tocando su límite (es decir, $g_j(x^*) < 0$), entonces su multiplicador μ_j forzosamente debe ser 0.